Prescription 53	When dealing with legacy code, respect code that works. You don't know what constraints the previous coders worked under.

Legacy code is scary—not because it's bad, but because you don't know what parts of the design are incidental and what parts are critical. When two different actions are backed by the same method but one of them has a bug, you don't know whether the other action is intended to have the current behavior.

When dealing with legacy code, testing is used for the following:

- Describing the current behavior of the code
- Making sure code changes have no unintended consequences
- Promoting isolation of new features

Set Expectations

You aren't going to convert this beast of an old project into a marvel of elegant, test-driven code overnight. And trying to do so is probably not a good idea. When you're exposed to a new codebase, you should do the following:

- If it makes you feel better, for 15 minutes shake your fists and curse the previous programmer's name (doubly effective if the previous programmer was you).
- Move on and start working with the codebase. It's yours now.

If you're like me, the temptation to immediately fix everything and/or add tests to everything is pretty strong. Lie down until the feeling passes.

There are two reasons it's a bad idea to do nothing but add test coverage to a legacy project first thing. From a purely logistical standpoint, when you take over a legacy project you're often expected to do something with it immediately. Your new client may not perceive going off in the corner and doing nothing but writing tests for weeks at a time as forward motion. Every situation is different, but it's rare to find a client that considers test coverage a "quick win."

The second problem has to do with the often-noted paradox of legacy development. Legacy code, by its nature, is often too interdependent or poorly understood to make it easy to unit-test without substantial refactoring. However, substantial refactoring without unit tests is a great way to introduce bugs into the codebase—especially when working with new code that you may not fully understand. This is also unlikely to be considered a "quick win."